

Week 2: Product and Category Management System

Scenario:

The railway reservation system wants to maintain information about **trains, passengers, stations, and reservations**. Each train operates between specific stations according to a scheduled journey. Passengers can search for available trains, check seat availability, and reserve tickets for their journey. The system should manage reservation details such as **passenger information, train details, journey date, seat number, coach, fare, and booking status**. It should also maintain payment and ticket information to ensure efficient and accurate reservation management.

1. Design Product and Category Tables

Category Table:

```
SQL> CREATE TABLE Category(  
 2      Category_ID VARCHAR2(8),  
 3      Category_Name VARCHAR2(15),  
 4      Description VARCHAR2(50)  
 5  );
```

Table created.

Product Table:

```
SQL> CREATE TABLE Product(  
2     Product_ID VARCHAR2(6),  
3     Product_Name VARCHAR2(15),  
4     Category_ID VARCHAR2(6),  
5     Price NUMBER(9,2),  
6     Stock_Quantity NUMBER  
7 );
```

Table created.

Train Table:

The two tables for **Train** and **Station** have been created. Now, we have to find the relationship between the tables and apply appropriate constraints to the tables. Each train operates between specific stations, and the system should maintain train details such as **train number, train name, source station, destination station, and train type**. The relationship between the Train and Station tables helps the system manage train routes and station information efficiently.

2. Define Relationships and Constraints

Relationships

One Train can have many Reservations.

The relationship between **Train** and **Reservation** is **One-to-Many** ($1 \rightarrow *$).

We have identified the relationship between the tables. Now we need to determine which keys are used to establish this relationship.

Before that, we need to understand the different **Key Constraints**.

PRIMARY KEY

The **Primary Key** is a column or field that uniquely identifies each row (tuple) in a table.

In these tables, which fields can be Primary Keys?

- In the **Train** table, **Train_ID** is a unique value that identifies each train.
- In the **Reservation** table, **Reservation_ID** is a unique value that identifies each reservation.

Therefore:

Train Table → **Train_ID** (Primary Key)

Reservation Table → **Reservation_ID** (Primary Key)

FOREIGN KEY

The **Foreign Key** is the key that links two tables.

Therefore, **Train_ID** acts as a **Foreign Key** in the **Reservation** table. It connects the **Train** table with the **Reservation** table.

Other Constraints

We can explore some other constraints such as **NOT NULL**, **CHECK**, and **UNIQUE**.

NOT NULL

The **NOT NULL** constraint ensures that a column cannot contain an empty (NULL) value.

For example, a **Train Name** or **Train Number** cannot be empty. Every train must have a name and train number.

CHECK

The **CHECK** constraint validates whether the value entered in a column satisfies a specified condition.

For example, we can check that the **Passenger Age must be greater than 0** and the **Ticket Fare must be a positive number**.

UNIQUE

The **UNIQUE** constraint ensures that there are no repeated values in a column.

For example, if we apply a **UNIQUE constraint to Train Number**, no two trains can have the same train number.

Defining Relationship and Creating Constraints for Product and Category:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT PK_Category
  3  PRIMARY KEY (Category_ID);
```

Table altered.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT PK_Product
  3  PRIMARY KEY (Product_ID);
```

Creating Relationship using FOREIGN KEY:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY (Category_ID)
  4  REFERENCES Category(Category_ID);
```

Table altered.

Adding Constraints: NOT NULL:

```
SQL> ALTER TABLE Category
  2  MODIFY Category_Name VARCHAR2(100) NOT NULL;

Table altered.

SQL> ALTER TABLE Product
  2  MODIFY Product_Name VARCHAR2(100) NOT NULL;

Table altered.
```

CHECK:

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Price
  3  CHECK (Price > 0);

Table altered.

SQL> ALTER TABLE Product
  2  ADD CONSTRAINT CHK_Stock
  3  CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE:

```
SQL> ALTER TABLE Category
  2  ADD CONSTRAINT UQ_Category_Name
  3  UNIQUE (Category_Name);

Table altered.
```

3. Insert Values:

Table altered.

```
SQL> INSERT INTO Category
  2  VALUES(
  3      '&Category_ID',
  4      '&Category_Name',
  5      '&Description'
  6  );
Enter value for category_id: CAT00001
old 3:      '&Category_ID',
new 3:      'CAT00001',
Enter value for category_name: Electronics
old 4:      '&Category_Name',
new 4:      'Electronics',
Enter value for description: Electronic Gadget and devices
old 5:      '&Description'
new 5:      'Electronic Gadget and devices'
```

1 row created.

```
SQL> /
Enter value for category_id: CAT00002
old 3:      '&Category_ID',
new 3:      'CAT00002',
Enter value for category_name: Colthing
old 4:      '&Category_Name',
new 4:      'Colthing',
Enter value for description: men,women and kids apparels
old 5:      '&Description'
new 5:      'men,women and kids apparels'
```

1 row created.

```
SQL> /
Enter value for category_id: CAT00003
old 3:      '&Category_ID',
new 3:      'CAT00003',
Enter value for category_name: Stationary
old 4:      '&Category_Name',
new 4:      'Stationary',
Enter value for description: notebook,pens and papers
```

Product Table:

```

SQL> INSERT INTO Product
2  VALUES(
3      '&Product_ID',
4      '&Product_Name',
5      '&Category_ID',
6      &Price,
7      &Stock_Quantity
8  );
Enter value for product_id: p10001
old 3:      '&Product_ID',
new 3:      'p10001',
Enter value for product_name: smartphone
old 4:      '&Product_Name',
new 4:      'smartphone',
Enter value for category_id: CAT002
old 5:      '&Category_ID',
new 5:      'CAT002',
Enter value for price: 25000
old 6:      &Price,
new 6:      25000,
Enter value for stock_quantity: 20
old 7:      &Stock_Quantity
new 7:      20

1 row created.

SQL> /
Enter value for product_id: P10002
old 3:      '&Product_ID',
new 3:      'P10002',
Enter value for product_name: Laptop
old 4:      '&Product_Name',
new 4:      'Laptop',
Enter value for category_id: CAT002
old 5:      '&Category_ID',
new 5:      'CAT002',
Enter value for price: 55000
old 6:      &Price,
new 6:      55000,

```

Like this insert few more rows.

After Insertion Category Table:

```
SQL> SELECT * FROM Category;
```

```
CATEGORY
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
CAT001
```

```
Silicone
```

```
Soft silicone phone cases
```

```
CAT002
```

```
Leather
```

```
Premium leather phone cases
```

```
CATEGORY
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
CAT003
```

```
Transparent
```

```
Clear transparent phone cases
```

```
CAT004
```

```
Printed
```

```
CATEGORY
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

Customized printed phone cases

CAT005

Rugged

Strong protective phone cases

CAT00001

CATEGORY

CATEGORY_NAME

DESCRIPTION

Electronics

Electronic Gadget and devices

CAT00002

Colthing

men,women and kids apparels

CATEGORY

CATEGORY_NAME

DESCRIPTION

CAT00003

Stationary

notebook,pens and papers

CAT00004

Home Appliances

Home essentials and needs

CATEGORY

CATEGORY_NAME

After Insertion Product Table:

PRODUC			

PRODUCT_NAME			

CATEGORY	PRICE	STOCK_QUANTITY	
-----	-----	-----	
Camera			
CAT00005	4500	16	
P1007			
Charging pin			
CAT00005	500	45	
PRODUC			

PRODUCT_NAME			

CATEGORY	PRICE	STOCK_QUANTITY	
-----	-----	-----	
P1008			
Species			
CAT00006	100	60	
P1009			
Utencils			
CAT00006	1000	55	
PRODUC			

PRODUCT_NAME			

CATEGORY	PRICE	STOCK_QUANTITY	
-----	-----	-----	
9 rows selected.			

CRUD Operations:

Insert Record:

```
SQL> INSERT INTO Product (Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
2  VALUES ('P10004', 'Tablet', 'CAT00002', 10000, 10);

1 row created.
```

Update Price:

```
SQL> UPDATE Product SET Price = 76000 WHERE Product_Name = 'Laptops';

1 row updated.
```

Delete a Product:

```
SQL> DELETE FROM Product WHERE Product_Name = 'Laptops';

1 row deleted.
```

5. Generate Category-wise Product Reports

Write SQL queries to:

- Display all products under each category.

```
SQL> SELECT c.CATEGORY_ID,  
2         c.CATEGORY_NAME,  
3         p.PRODUCT_ID,  
4         p.PRODUCT_NAME,  
5         p.PRICE,  
6         p.STOCK_QUANTITY  
7 FROM Category c  
8 JOIN Product p  
9 ON c.CATEGORY_ID = p.CATEGORY_ID  
10 ORDER BY c.CATEGORY_ID;
```

```
CATEGORY  
-----  
CATEGORY_NAME  
-----  
PRODUCT  
-----  
PRODUCT_NAME  
-----  
PRICE STOCK_QUANTITY  
-----  
CAT00002  
Clothing  
P10002  
CATEGORY  
-----  
CATEGORY_NAME  
-----  
PRODUCT  
-----  
PRODUCT_NAME  
-----  
PRICE STOCK_QUANTITY  
-----  
Croptops and Shirt  
2000 35
```

```
PRODUCT  
-----  
PRODUCT_NAME  
-----  
PRICE STOCK_QUANTITY  
-----  
Spices  
100 60  
CATEGORY  
-----  
CATEGORY_NAME  
-----  
PRODUCT  
-----  
PRODUCT_NAME  
-----  
PRICE STOCK_QUANTITY  
-----  
CAT00006  
Provisions  
P1009  
CATEGORY  
-----  
CATEGORY_NAME  
-----  
PRODUCT  
-----  
PRODUCT_NAME  
-----  
PRICE STOCK_QUANTITY  
-----  
Utencils  
1000 55  
9 rows selected.
```

Active
Go to 5

- Count number of products in each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_COUNT
```

```
-----
```

```
Clothing
```

```
2
```

```
Stationary
```

```
1
```

```
Home Appliances
```

```
2
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_COUNT
```

```
-----
```

```
Mobile Parts
```

```
2
```

```
Provisions
```

```
2
```

: Find highest-priced product in each category.

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----
```

```
AVG_PRICE
```

```
-----
```

```
Clothing
```

```
6000
```

```
Stationary
```

```
50
```

```
Home Appliances
```

```
14000
```

```
CATEGORY_NAME
```

```
-----
```

```
AVG_PRICE
```

```
-----
```

```
Mobile Parts
```

```
2500
```

```
Provisions
```

```
550
```

Display categories having more than 5 products.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_COUNT
```

```
-----
```

```
Clothing
```

```
2
```

```
Stationary
```

```
1
```

```
Home Appliances
```

```
2
```

```
CATEGORY_NAME
```

```
-----
```

```
PRODUCT_COUNT
```

```
-----
```

```
Mobile Parts
```

```
2
```

```
Provisions
```

```
2
```

- Find average product price category-wise

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
2 FROM Product p
3 JOIN Category c ON p.Category_ID = c.Category_ID
4 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
-----
```

```
AVG_PRICE
```

```
-----
```

```
Clothing
```

```
6000
```

```
Stationary
```

```
50
```

```
Home Appliances
```

```
14000
```

```
CATEGORY_NAME
```

```
-----
```

```
AVG_PRICE
```

```
-----
```

```
Mobile Parts
```

```
2500
```

```
Provisions
```

```
550
```


